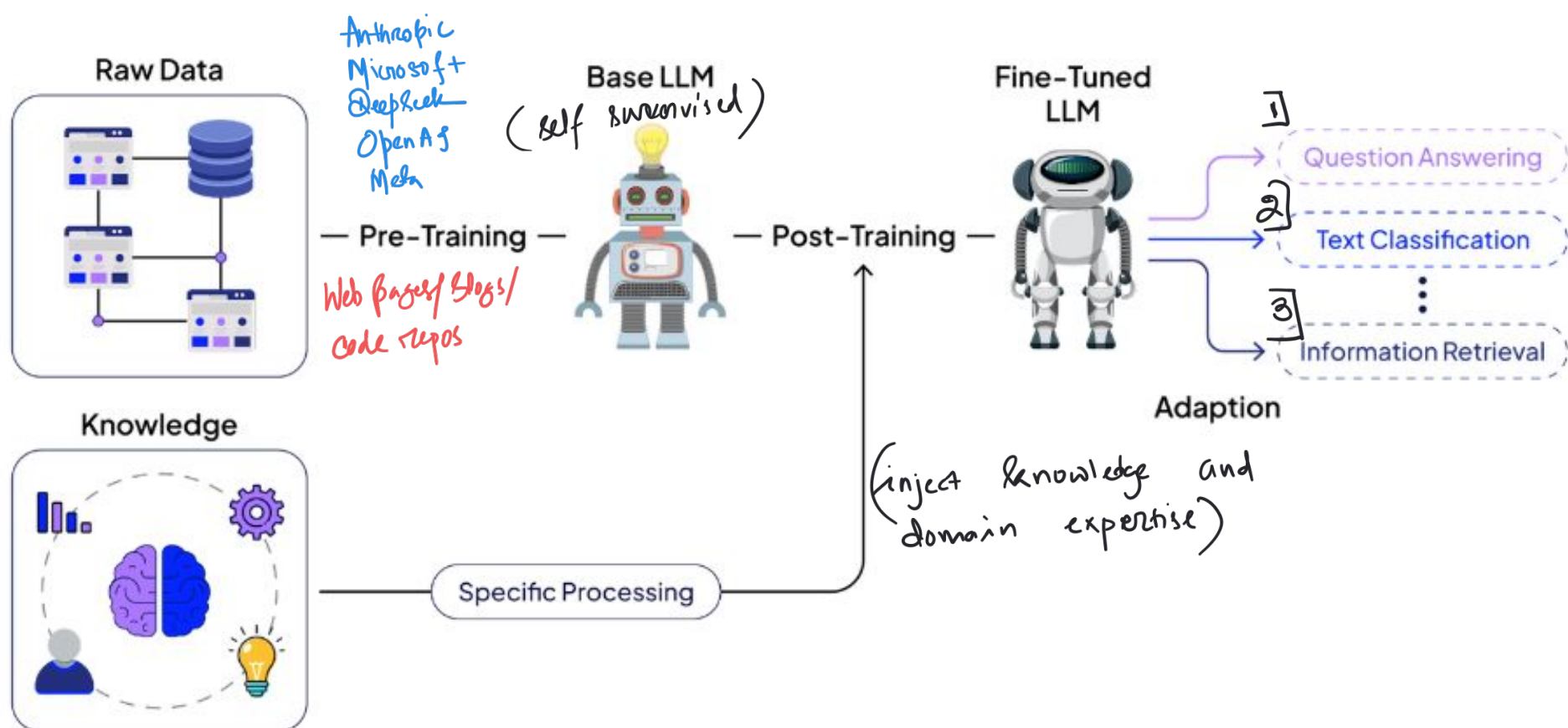


Fine Tuning Existing Models



y: man
 ↑
 (LLM)
 ↑

x: I am a good man.

} Self supervised
pretraining

(GPT 2/3
Llama
BERT

Next token prediction loss

x: New Delhi is the capital of
y: India ✓

x		y
New Delhi is capital of		India

Base LLMs

- understand a language
- learn grammar
- world knowledge
- reasoning patterns

Post training

Human Annotators
GPT 4.6

Supervised

1] Instruction finetuning

✓

X:

Instruction: Summarize the article — W1 Article

W20 Article

{

y:

Response: — — —

Actual Response

Microsoft Phi

2] SFT

x:

User: Explain Gradient Descent

y:

Assistant: Gradient Descent is an optimization problem

large

X: { The capital of France is ~~Paris~~ }
y: { Paris }

large
X: { Instruction: Summarize the article [. . .] }
y: { Summary }

- Filtering
- Synthetic data
- Mixing

- Q&A format
- Long-context stage
- Continued pre-training
- High-quality stage
- Knowledge distillation

next token prediction

- Supervised finetuning (SFT)
- Reinforcement learning with human feedback (RLHF)
- Direct preference optimization (DPO)
- Online/offline
- Knowledge distillation



Dataset

Web text
Books
Wiki



Preprocessing



Pre-training



Post-training



Optimization



X: Sentence 1
y: true
X: Sentence 2
y: -ve

1] GPT 3

2] Can you translate a sentence
to German

3] RLHF

Use **RLHF** or **DPO** here

Pretraining

Instruction finetuning

Alignment

Reinforcement Learning

A typical LLM development flow

Fine-tuning libraries



Unisloth

More beginner-friendly and guided
with many additional features



TRL

Most up-to-date and customizable
for research projects

For each parameter
TRAINING

B = Bytes

	Bytes per parameter
Model Parameters (Weights)	4 bytes per parameter
Adam optimizer (2 states)	+8 bytes per parameter
Gradients	+4 bytes per parameter
Activations and temp memory (variable size)	+8 bytes per parameter (high-end estimate)
TOTAL	=4 bytes per parameter +20 extra bytes per parameter

4B/parameter

(4B + 4B) / parameter

4B/parameter

} 24B/parameter
24 Bytes



Llama 8B Model

GPU VRAM

needed for training /

inference

no. of params is 8B model = 8B

$$\underline{\underline{8 \times 10^9 \times 24 \text{ Bytes}}}$$

8 Billion

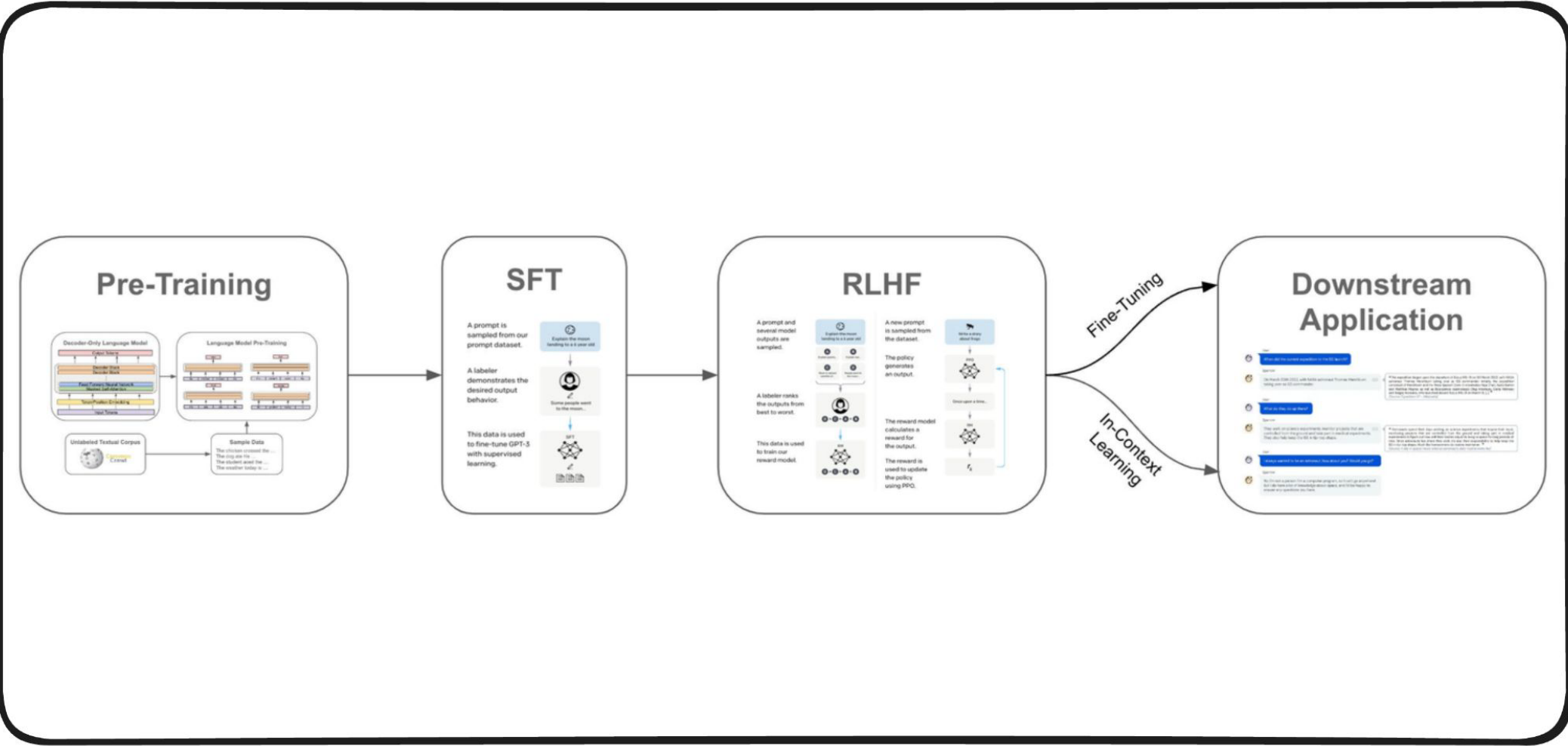
= 8×10^9 parameters

$= 8 \times 10^9 \times 24 \text{ Bytes}$

Model +



VRAM

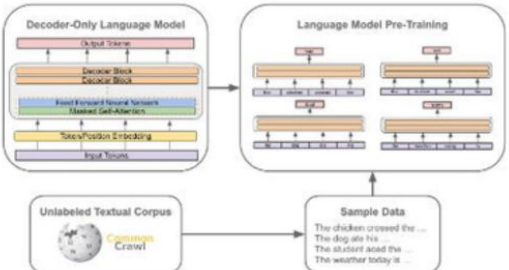


Alignment

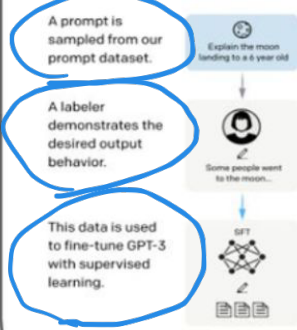
Self supervised ✓

Supervised

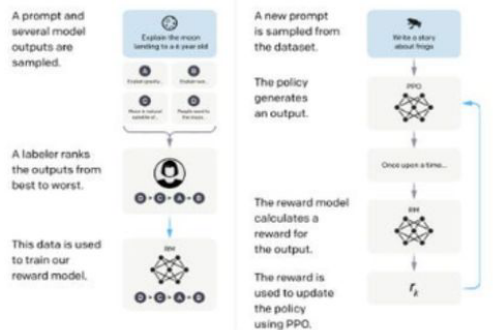
Pre-Training



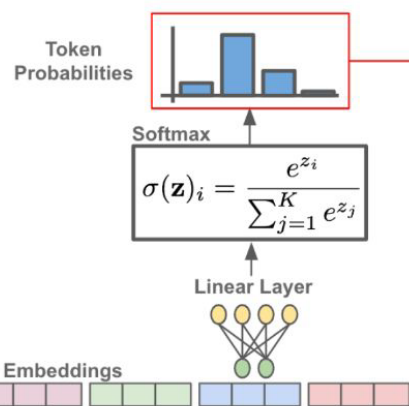
SFT



RLHF



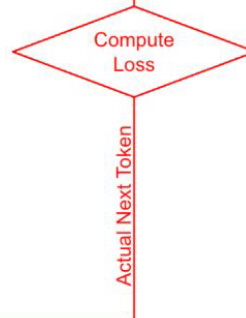
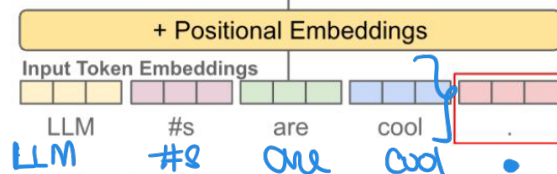
y: size of vocab



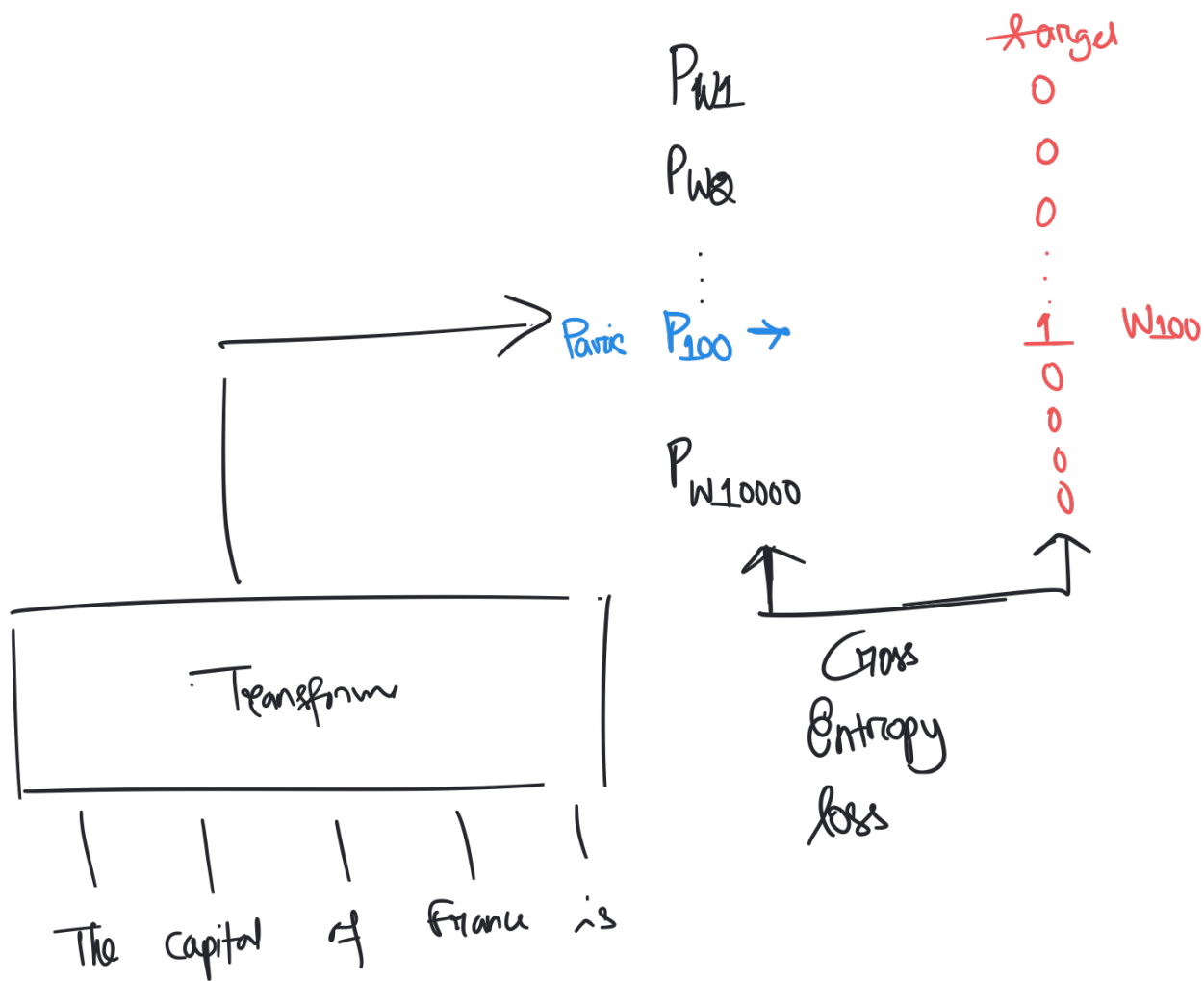
Find layer softmax - how many neurons?

Transform

x: {



Next
Token
Prediction



~~The capital of France is~~ Paris

Annotated Datasets for Finetuning

X:	► Prompt:	Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.
	Response:	Hydrogen comes first as element number one. Helium is second for balloons to have fun! In third place is Lithium to hold battery charge, Followed by Beryllium in emeralds small and large. Boron's number five to help us keep things clean. Carbon's next at six, and it's found in every gene. Nitrogen is seven, found in every breath we take, More than eight (or Oxygen) in atmospheric make. Number nine is Fluorine, helping polish up our teeth. Neon gives us glowing signs to read and stand beneath.
y:		
X:	► Prompt:	I want you to roast me. I want you to make it particularly brutal, swearing at me.
	Response:	I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.
y:		

Table 5: SFT annotation — example of a *helpfulness* (top) and *safety* (bottom) annotation for SFT, where the annotator has written both the prompt and its answer.

~~Self~~
Pretraining

Wiki | The capital of India is | Delhi |

X: The capital of India is

y: Delhi

Finetuning

X: Can you solve the equation $5x + 2 =$
y: 7

X: Can you summarize this doc
(w1 doc1, ...)

y: summarized

weight updates

final

Finetuned Weights

Weight Update

$$\overbrace{W_{ft}} = \underbrace{\overset{\text{don't have gradient}}{W_{pt}}} + \overbrace{\Delta W_{\underline{A \cdot B}}}$$

Pretrained Weights

initial

In full fine-tuning, ΔW is an unconstrained $d \times k$ matrix — the same size as W_0 . LoRA's key insight is to **constrain** ΔW to be **low-rank** by decomposing it into two smaller matrices:

$$W' = W_0 + \frac{\alpha}{r} \cdot BA$$

Where: $\mathbb{R}^{d \times k}$ $\mathbb{R}^{1000 \times 1000}$

- $W_0 \in \mathbb{R}^{d \times k}$ — **frozen** pretrained weights } no grad / no adam
 - $B \in \mathbb{R}^{d \times r}$ — trainable "up-projection" matrix → $B_{10 \times 1000}$
 - $A \in \mathbb{R}^{r \times k}$ — trainable "down-projection" matrix → $A_{1000 \times 10}$
 - $r \ll \min(d, k)$ — the **rank** (typically 4–64) = 10
 - α — scaling hyperparameter
 - $\frac{\alpha}{r}$ — effective scaling factor
- } } gradient / adam trainable

$$W_{1000 \times 1000} = \overset{\text{Init}}{W_{1000 \times 1000}} + \overset{\text{learn}}{\frac{\alpha}{\eta}} A \cdot B$$

$\frac{\alpha}{\eta}$ A B
 4 1000×4 4×1000

$$W = \overset{\text{init}}{W} + \frac{\alpha}{\eta} A \cdot B$$

$\frac{\alpha}{\eta}$ A B
 64 1000×64 64×1000

(more)

effective update strength = $\frac{\alpha}{\eta}$

magnitude of update $\propto \text{rank}(\eta)$

$$\Delta W = A \cdot B$$

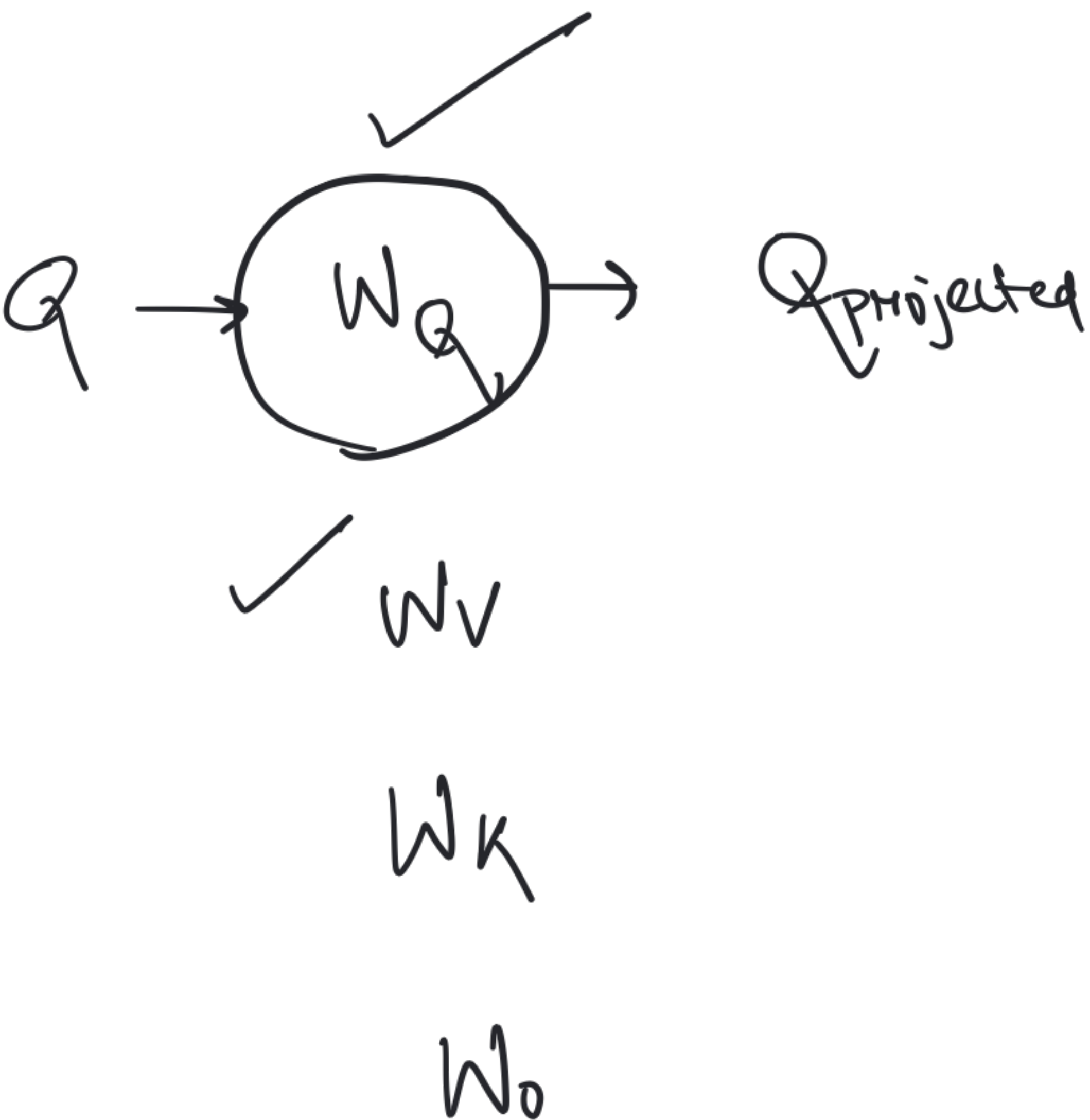
$$W_{\text{pretrained}} + \alpha \frac{\Delta W}{\eta}$$

increase $\Rightarrow$ increase catastrophic forgetting

decreasing α

decrease

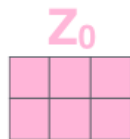
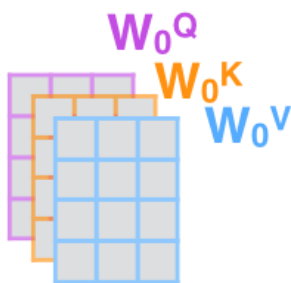
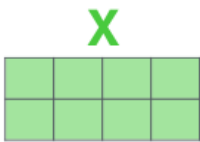
decrease catastrophic forgetting



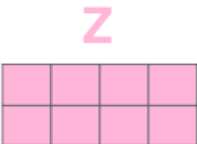
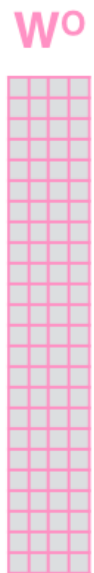
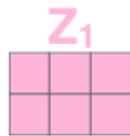
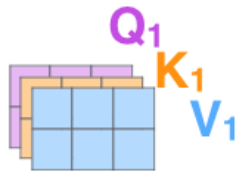
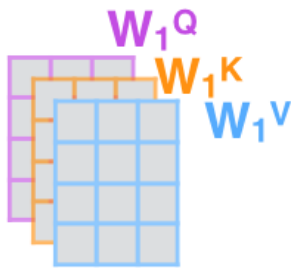
$$W^Q \quad W^K \quad W^V$$

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

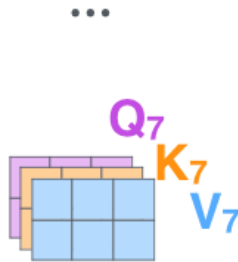
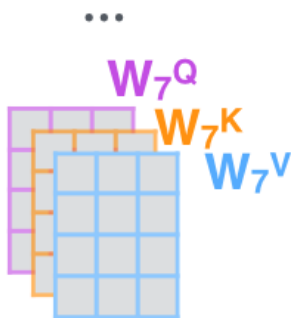
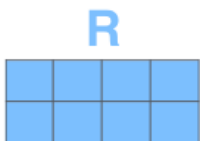
Thinking Machines



$$W^O$$



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

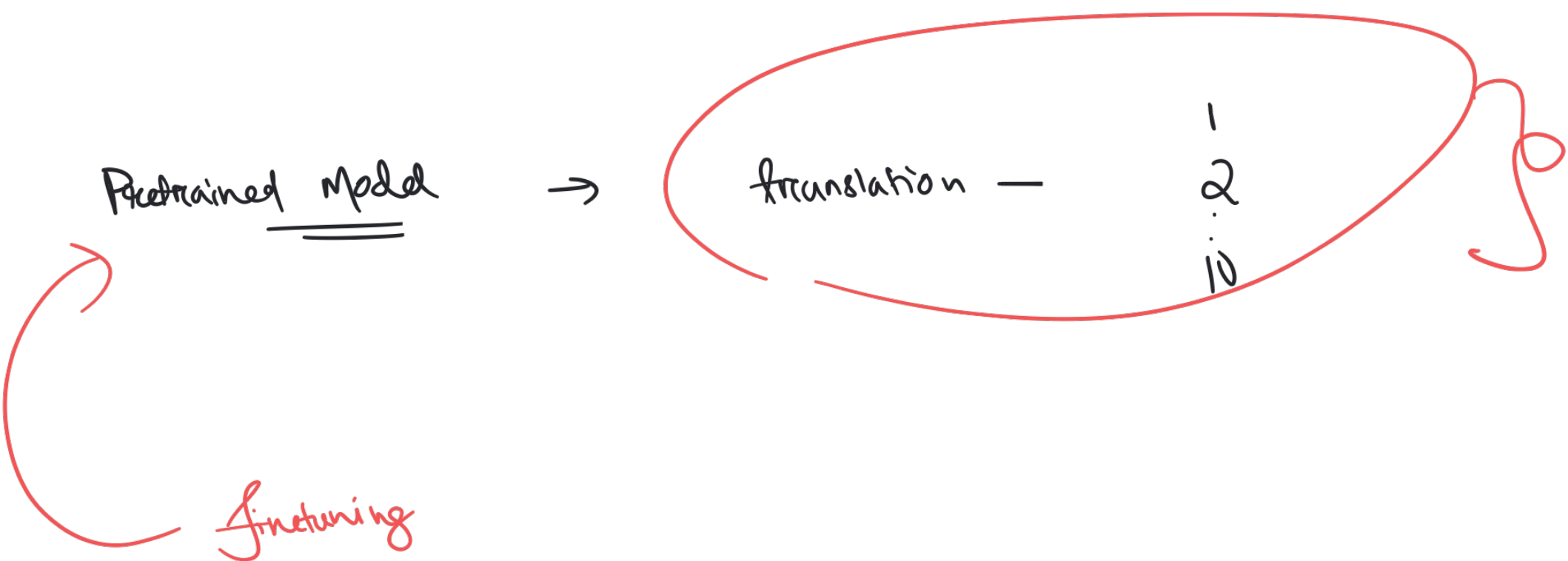


$$W^u = W^{\text{initial}} + \frac{\alpha}{n} \left(\begin{array}{cc} \text{A.} & \text{B} \end{array} \right) \begin{array}{l} \rightarrow 0 \\ 55 \\ 0 \end{array}$$

N_1 N_2
 $\swarrow$ $\searrow$
 0

0

updates = 0



✓
Pretrain + finetune ————— same 10 tasks

updates are low — increased α
updates are too much — decrease α



Control

Catastrophic forgetting — Pretrained LM = W_{init}

↑
make too much of
changes to W_{init} ,
it might forget
some initial
data patterns

Forward pass

no gradients

$$y = \left(W + \frac{\alpha}{\eta} B \cdot A \right) x$$

gradients

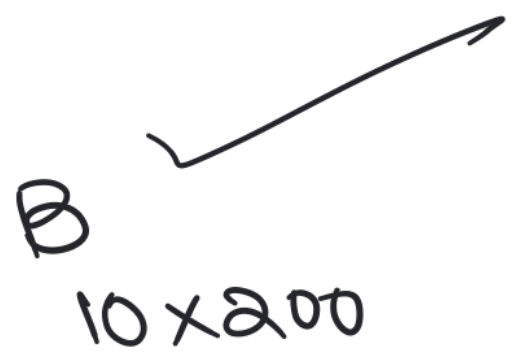
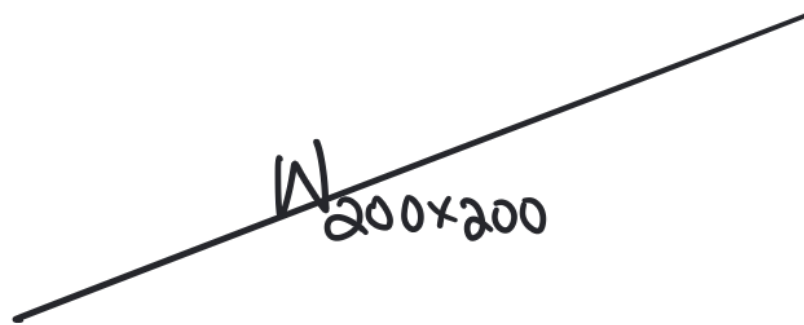
$$y = Wx$$

Backward pass

10000 → []

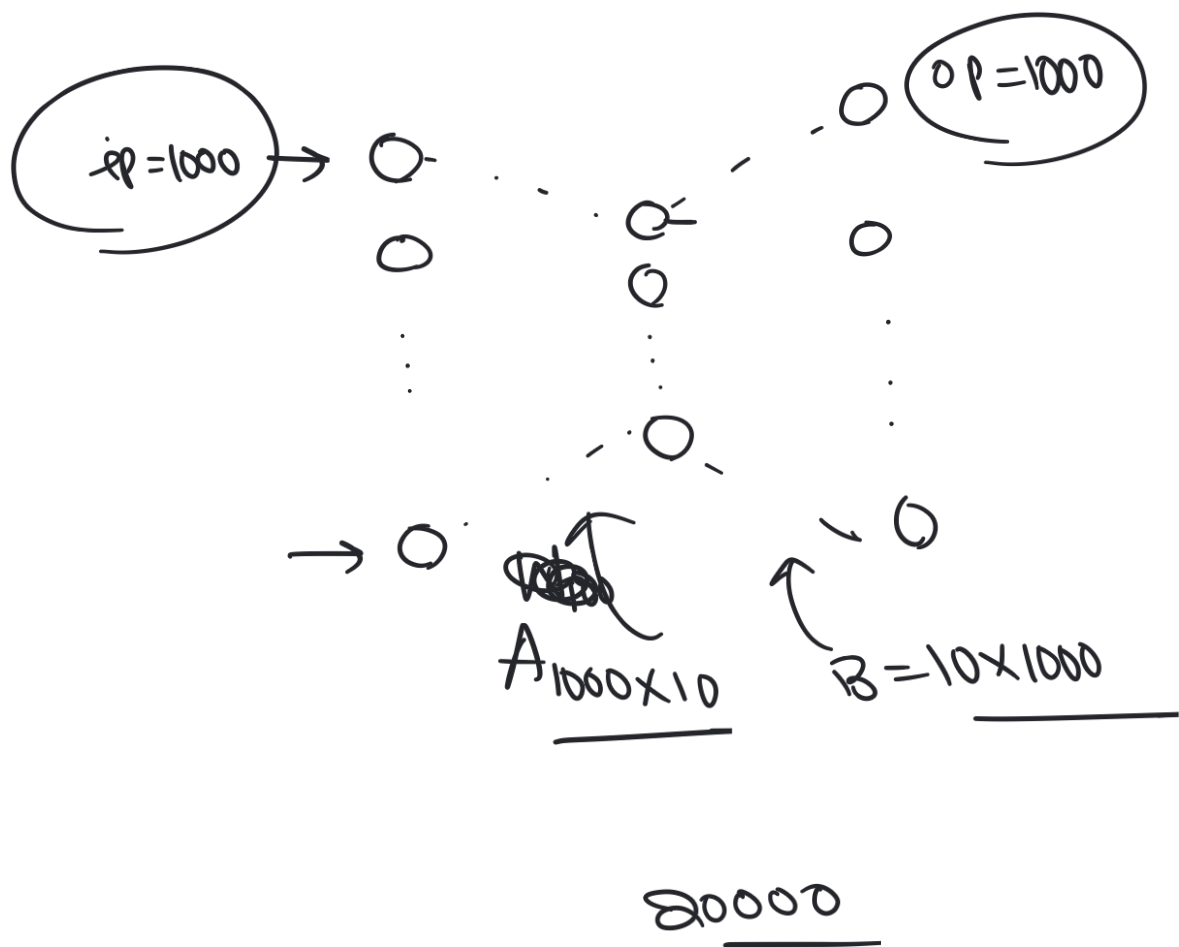
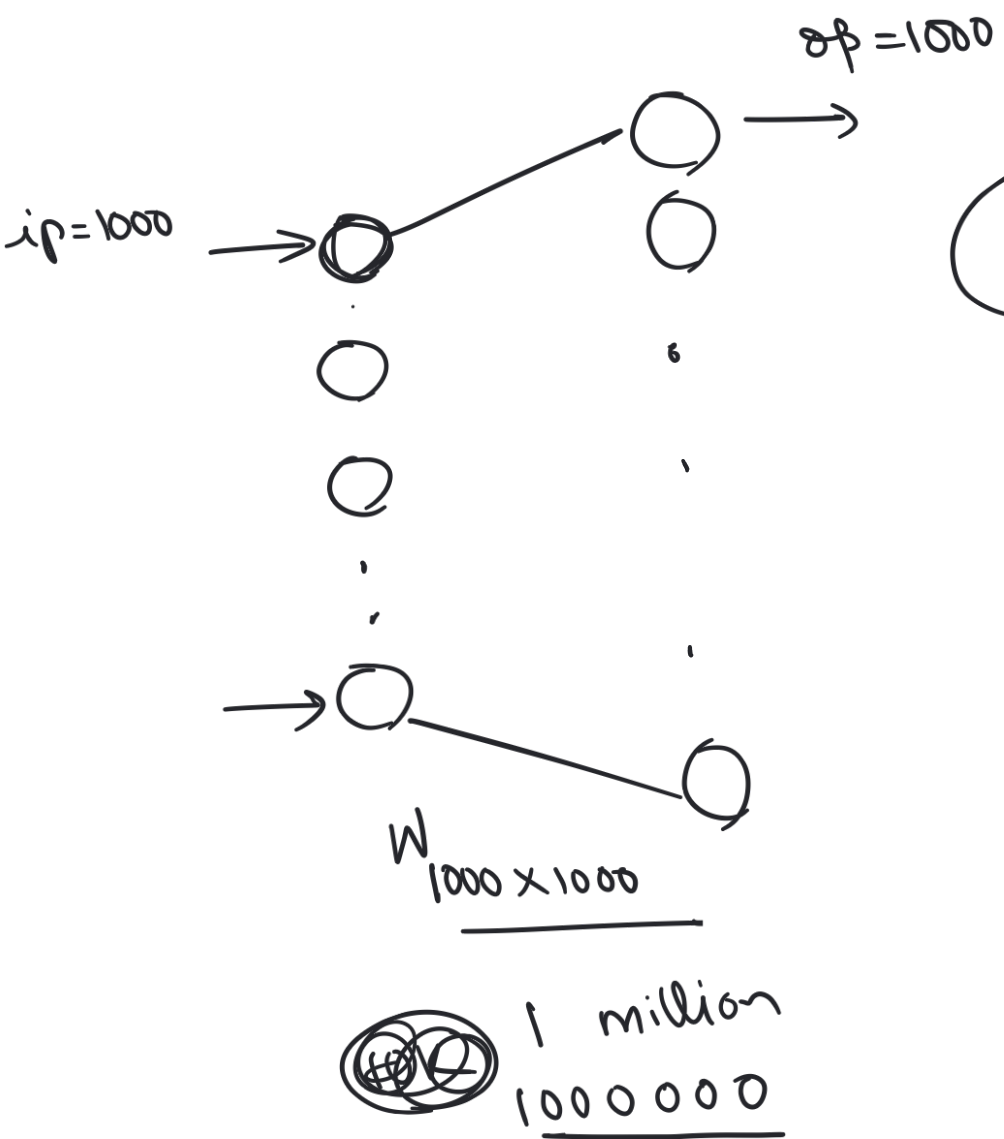
B · A

$$200 \times 200 = 40000$$



rank = 10
↑

↓
 $D_1(200, 10)$
 $D_2(10, 200)$



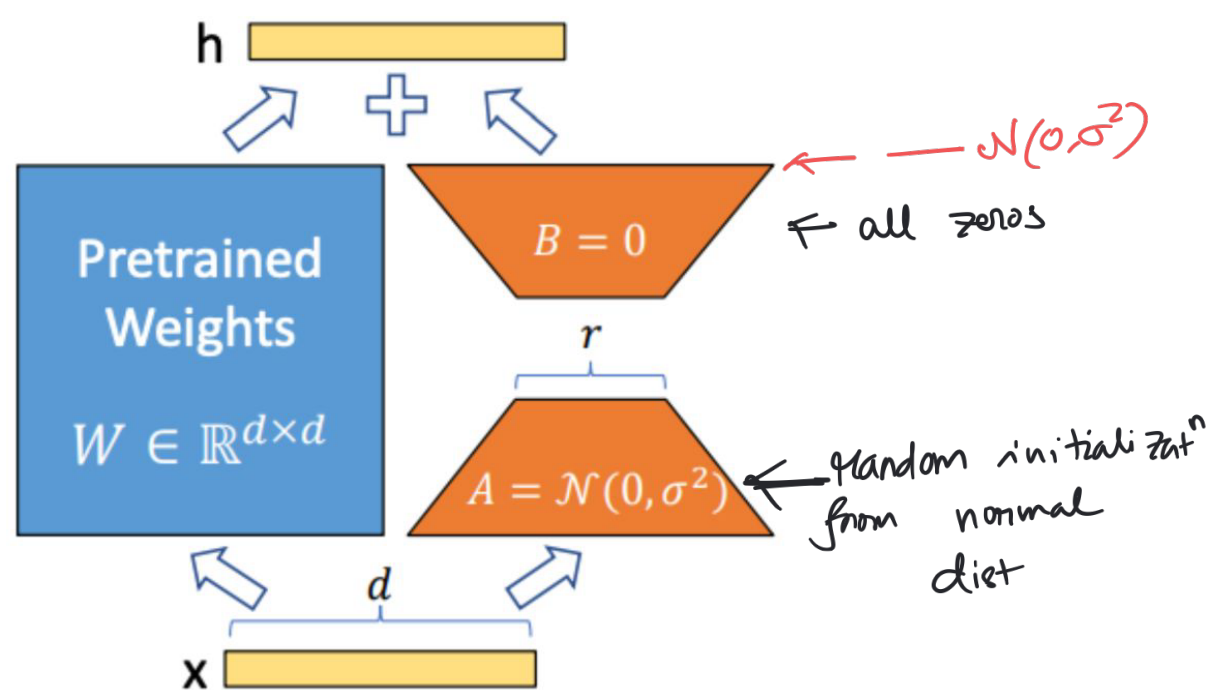


Figure 1: Our reparametrization. We only train A and B .

Rank Decomposition
Matrix

$$W_{\text{ft}} = W_{\text{pt}} + \underbrace{\Delta W}_{\text{Approximation}} = W_{\text{pt}} + \underbrace{AB}_{\text{Rank Decomposition Matrix}}$$

where $W_{\text{ft}}, W_{\text{pt}}, \Delta W, AB \in \mathbb{R}^{d \times d}$

and $\underbrace{A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}}_{\text{Low Rank}}$

$$W = \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix}$$

$$W_{\text{finetuned}} = \underline{\underline{W + \Delta W}}$$

$$\text{Full finetuning} = \Delta W$$

$$\underline{\underline{25}}$$

→ instead of learning the full 4×4 update

$$\Delta W = A \cdot B$$

$$\begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix} \cdot \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix}$$

$$\begin{aligned} 5 \times 2 &= 10 \\ 2 \times 5 &= 10 \\ \underline{\underline{20}} \end{aligned}$$

W_{init}
 1000×1000

$W_{finetune}$
 1000×1000



ΔW
 1000×1000

1000000

$$\Delta W = \left(A_{1000 \times 10} \cdot B_{10 \times 1000} \right)$$

$(1000 \times 10 + 10 \times 1000) = 20000$

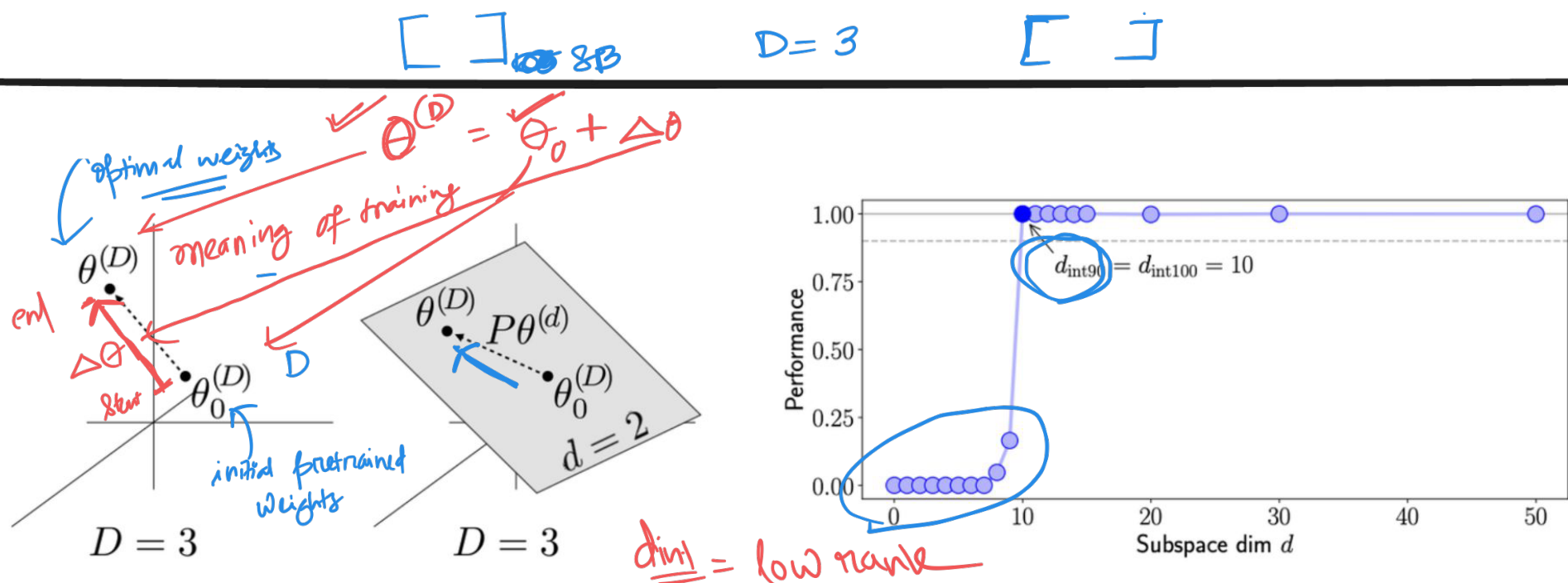


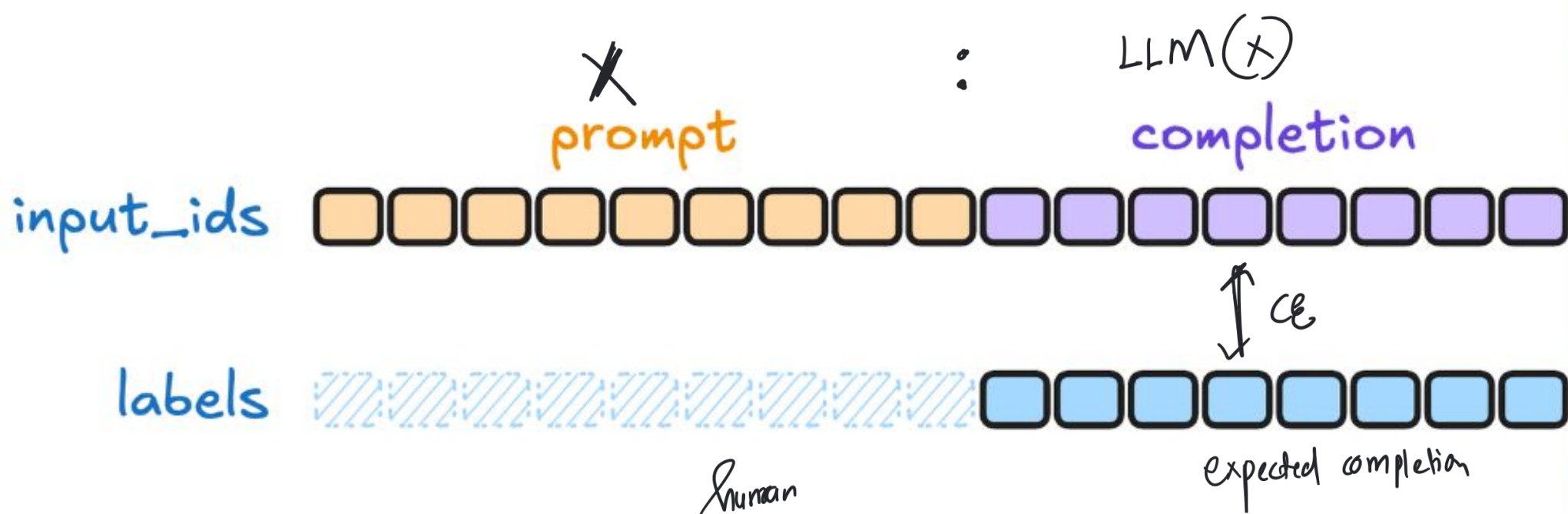
Figure 1: **(left)** Illustration of parameter vectors for direct optimization in the $D=3$ case. **(middle)** Illustration of parameter vectors and a possible random subspace for the $D=3, d=2$ case. **(right)** Plot of performance vs. subspace dimension for the toy example of toy example of Sec. 2. The problem becomes both 90% solvable and 100% solvable at random subspace dimension 10, so $d_{\text{int}90}$ and $d_{\text{int}100}$ are 10.

pretrained
initial wts
 Θ_0
wts after
finetuning
 Θ

$$\Theta = \Theta_0 + \Delta\theta$$

|
train / finetune

$$\Theta = \Theta_0 + \underbrace{AB}_{\text{lower subspace dim}}$$



Instruction: What is the capital of France? Prompt
 Response: The capital of France is Paris

$P(p_{\text{prompt}}) \rightarrow$

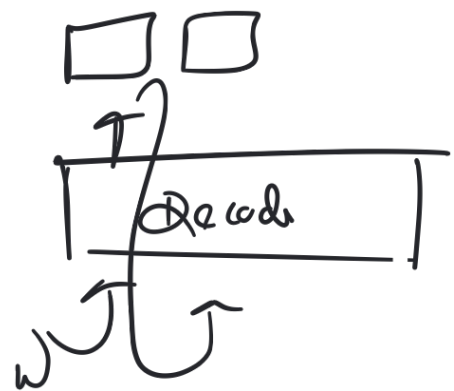
<u>pred</u>	<u>target</u>
p_1	0
p_2	0
$\vdots$	$\vdots$
p_{10000}	1
	0
	0

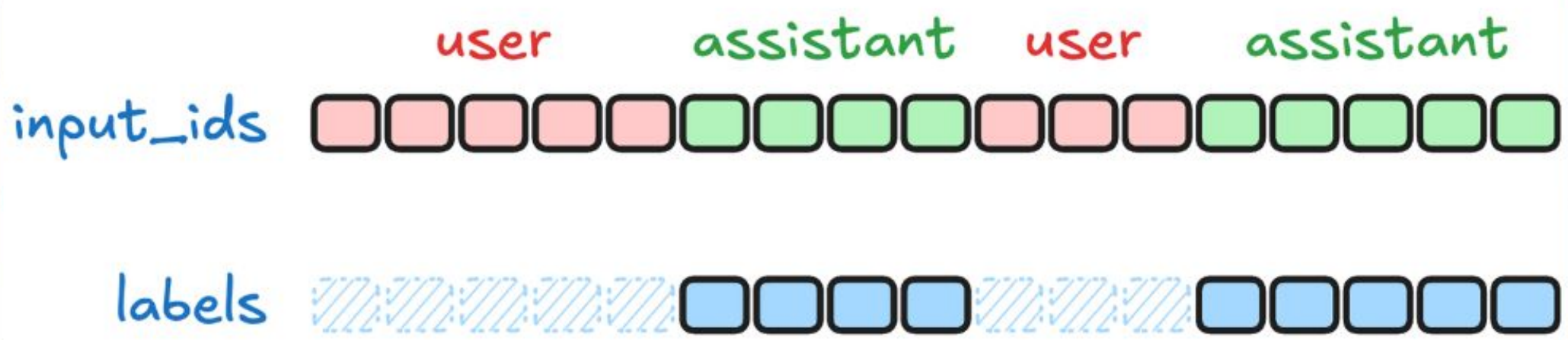
 the word

$P(p_{\text{prompt}}, x_{\text{he}}) \rightarrow$

<u>pred</u>	<u>target</u>
p_1	0
p_2	0
p_3	0
$\vdots$	$\vdots$
p_{10000}	1
	0
	0
	0

 capital





epoch 1

$$W^{\text{update}} = \overbrace{W^{\text{initial}}} + \underbrace{A \cdot B}_{\substack{\uparrow \\ 0}} \quad \underbrace{\begin{matrix} = 0 \\ \sum 0 \end{matrix}}$$